CyberSecIntel
P2 Follow-Up Deliverable 2.2
Basic Implementation

Dídac Cayuela and Sindri Masson
Big Data Management – Universitat Politecnica de Catalunya

May 2026

1 Instructions

This follow-up implements the basic data movement required by P2. The objective is not to finalize every transformation, but to prove that the architecture from Deliverable 2.1 is executable end to end: P1 landing/silver Delta tables are read, Trusted Zone checkpoints are written, Exploitation Zone assets are materialized, and consumption files are exported for analysts.

The implementation uses the same Option 1 architecture selected in P2.1. MinIO and Delta Lake remain the source of truth for all zones. The new buckets are `s3://trusted/` and `s3://exploitation/`; the original P1 Delta bucket `s3://deltalake/` is unchanged.

Run order.

1. Start the stack: `docker compose up -d minio zookeeper kafka postgres`, then initialize and start Airflow.
2. Run the P1 ingestion DAGs: `cybersecintel\_api\_expansion\_ingestion`; optionally run `cybersecintel\_dataset\_artifact\_ingestion` and `cybersecintel\_pcap\_replay`.
3. Trigger `cybersecintel\_trusted\_zone`.
4. Trigger `cybersecintel\_exploitation\_zone`.
5. Trigger `cybersecintel\_consumption\_exports`.

The same stages can be run without Airflow:

```
python -m trusted.run_trusted
python -m exploitation.run_exploitation --warm
python -m consumption.run_exports
```

2 Architecture Design

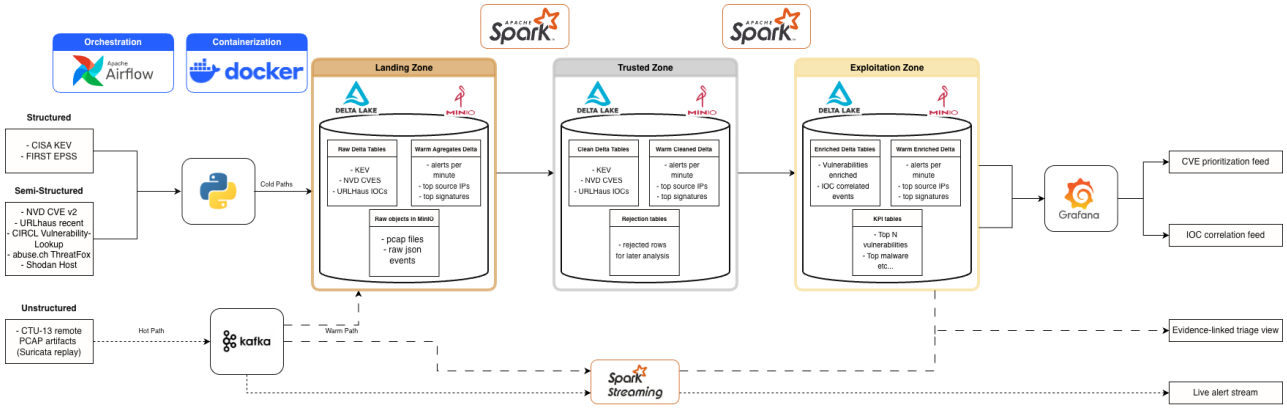

Figure 1: P2 architecture implemented in this follow-up. P2.2 focuses on executable zone checkpoints and basic consumption outputs.

Component	P2.2 implementation
Trusted Zone	Top-level `trusted/` package. Reads existing P1 Delta tables from `s3://deltalake/`, performs conservative cleaning, and writes cleaned and rejected rows to `s3://trusted/`.
Exploitation Zone	Top-level `exploitation/` package. Reads trusted tables and writes `vuln\_enriched`, `network\_events`, `ioc\_correlations`, `anomaly\_flags`, `warm\_enriched\_alerts`, and KPI tables to `s3://exploitation/`.
Consumption	Top-level `consumption/` package. Exports CSV, JSON, and HTML analyst artifacts under `consumption/outputs/`.
Orchestration	Three new Airflow DAGs: `cybersecintel\_trusted\_zone`, `cybersecintel\_exploitation\_zone`, and `cybersecintel\_consumption\_exports`.
Streaming	A bounded warm-path consumer reads a finite batch from Kafka topic `ids.alerts`, enriches alerts when CVE context is available, and writes `warm\_enriched\_alerts`.

3 Trusted Zone

The Trusted Zone applies information-preserving cleaning. The code does not remove analytical signals or perform task-specific modeling. It standardizes key fields, casts common numeric fields, normalizes CVE identifiers, validates basic dates and URLs, deduplicates by natural keys, and records quality warnings.

Rows that fail mandatory-field checks are written to `rejected\_*` tables. Duplicate rows are also preserved there with a rejection reason. This keeps the cleaning stage auditable while giving downstream tasks a cleaner checkpoint.

Source family	Basic trusted implementation
Structured	`kev` and `epss`: CVE normalization, mandatory-field checks, deduplication, date parsing, EPSS range warnings.
Semi-structured	`nvd`, `urlhaus`, `circl\_vulnlookup`, `threatfox`, and `shodan\_seeded`: stable helper fields, CVE or indicator normalization, basic URL and score warnings.
Network / unstructured-derived	`pcap\_artifacts`, `pcap\_replay\_runs`, `suricata\_events`, and `ids\_alerts`: timestamp validation, integer casting, deduplication, and ingest-date preservation.

4 Exploitation Zone

The Exploitation Zone materializes analyst-ready assets:

- `vuln\_enriched`: CVE-level table joining KEV, NVD, EPSS, and CIRCL by normalized CVE id.
- `network\_events`: unified schema for trusted IDS alerts and Suricata events.
- `ioc\_correlations`: matches URLhaus, ThreatFox, and Shodan indicators against observed network fields when possible.
- `anomaly\_flags`: rule-based P2.2 anomaly feed using severity, repeated source IPs, and suspicious destination ports.
- KPI tables: top CVEs by priority, daily alert counts, top signatures, top source IPs, and recent KEV additions.

The vulnerability priority score follows the P2.1 idea: combine EPSS likelihood, KEV active exploitation evidence, and CVSS severity when present. If CVSS is unavailable in the current P1 source snapshot, the score still remains usable through EPSS and KEV.

5 Data Consumption

The consumption stage writes concrete files under `consumption/outputs/`:

- `cve\_prioritization.csv`
- `ioc\_correlations.csv`
- `anomaly\_flags.csv`
- `alert\_trends.json`
- `soc\_dashboard.html`

Grafana is intentionally not introduced in this checkpoint. The current deliverable proves the data contracts and visible outputs first. A Grafana service can be added in the final P2 phase by mirroring selected KPI tables into a serving database while keeping Delta Lake as the authoritative source.

6 What Remains for Final P2

The final delivery should improve transformation depth and governance. The main remaining work is richer source-specific cleaning, stronger schema documentation, a more complete lineage table, optional Grafana provisioning, and replacing the rule-based anomaly feed with the Isolation Forest model proposed in P2.1 if time permits.

7 Validation

The implementation includes unit tests for trusted cleaning, exploitation joins and KPIs, warm-path enrichment, and consumption exports. Run:

```
.venv/bin/python -m unittest discover -s tests
```

For integration validation, inspect MinIO after running the DAGs and confirm that the `trusted` and `exploitation` buckets exist. Then open `consumption/outputs/soc\_dashboard.html` and verify that the generated CSV/JSON files exist.